

BİTİRME PROJESİ — GÜNCEL YOL HARİTASI

İZMİR ESHOT OTOBÜS SİSTEMİ

BAĞLAM DUYARLI DERİN ÖĞRENME İLE OTOBÜS VARIŞ SÜRESİ TAHMİNİ

Versiyon 2.0 — Tam GTFS Veri Seti Dahil Edilmiş

Muhatap Makale: Kaya & Kalay, IEEE Access 2025

Mart 2026

1. V2.0 GÜNCELLEME ÖZETİ

Bu güncellemede iki kritik GTFS dosyası (shapes.txt + stop_times.txt) ve projenin files bölümündeki dört ek GTFS dosyası (routes.txt, trips.txt, stops.txt, calendar.txt) sisteme dahil edilmiştir. Bu ekleme projenin metodolojisini kökten değiştiren önemli sonuçlar doğurmaktadır.

Dosya	Boyut	En Kritik Katkı
stop_times.txt ★	765.600 kayıt 22.934 sefer 10.348 durak	Planlanan duraklar arası süre → Gecikme hesabı mümkün
shapes.txt ★	350.858 nokta 788 şekil	Her seferin kesin güzergah geometrisi
trips.txt	67.397 sefer 394 hat	route_id ↔ trip_id ↔ shape_id bağlantısı
routes.txt	394 hat, tümü tip 3 (otobüs)	Hat adları ve kodları
stops.txt	10.483 durak enlem/boylam	Tüm durakların kesin koordinatları
calendar.txt	3 servis tipi	Hafta içi / Cumartesi / Pazar kesin ayrımı

1.1 Metodolojik Değişiklikler (V1.0 → V2.0)

Konu	V1.0 (Önceki Plan)	V2.0 (Güncel Plan)	Etki
Durak eşleşmesi	Haversine mesafe hesabı ile yakın durak bulma	stop_times'tan doğrudan durak ziyaret zamanı	Hata riski ortadan kalktı
Hedef değişken	Sadece gerçek seyahat süresi	Gecikme = gerçek – planlanan VEYA gerçek süre	Zenginleşti
Güzergah geometrisi	eshtotobushatguzergahlari.csv ile yaklaşık	shapes.txt ile kesin GPS yolu	Daha doğru eşleştirme
Gün tipi	Timestamp'ten türetilen yaklaşık sınıflandırma	calendar.txt'ten kesin: servis 2917/2926/2929	Makale ile birebir aynı
GPS eşleştirme	Tüm duraklar için kaba arama	Bilinen sefer–durak çiftine zamanlama yerleştirme	Çok daha verimli

2. TAM VERİ ENVANTERİ (V2.0)

2.1 GTFS Veri Seti — Artık Tam

GTFS (General Transit Feed Specification) standart bir toplu taşıma veri formatıdır. Projemizin GTFS seti artık eksiksizdir ve birbiriyle ilişkili 6 dosyadan oluşmaktadır:

GTFS Veri Modeli — İlişkisel Yapı	
routes.txt (394 hat) → trips.txt (67.397 sefer) → stop_times.txt (765.600 kayıt)	
↓ shape_id	shapes.txt (788 şekil, 350.858 nokta)
↓ service_id	calendar.txt (3 servis tipi: hafta içi / Cts / Paz)
	+ stops.txt (10.483 durak koordinatı)

Dosya	Kayıt Sayısı	Temel Kolonlar	Proje Rolü
routes.txt	394 hat	route_id, route_short_name, route_type(=3)	Hat meta verisi
trips.txt	67.397 sefer	route_id, service_id, trip_id, direction_id, shape_id	Sefer-hat-şekil köprüsü
stop_times.txt ★	765.600 kayıt	trip_id, arrival_time, departure_time, stop_id, stop_sequence	Planlanan zaman tabanı
stops.txt	10.483 durak	stop_id, stop_name, stop_lat, stop_lon	Durak koordinatları
shapes.txt ★	350.858 nokta	shape_id, shape_pt_lat, shape_pt_lon, shape_pt_sequence	Güzergah geometrisi
calendar.txt	3 servis	service_id, mon-sun (0/1), start/end_date	Gün tipi sınıflaması

2.2 stop_times.txt — Projenin Kritik Yeni Bileşeni

765.600 kayıt, 22.934 sefer (tümü hafta içi — service_id=2917). Bu dosyanın önemi:

stop_times Ne Sağlıyor?	Hesaplanan Temel İstatistikler
- Her seferin her durağa planlanan varış saati (arrival_time) - Her durağa planlanan kalkış saati (departure_time) - Bekleme süresi: departure – arrival = sabit 10 saniye - Durak sırası (stop_sequence: maks. 192) - Hafta içi sefer başına ortalama 33 durak ziyareti	742.666 geçerli duraklar arası segment - Ortalama planlanan seyahat süresi: 1.42 dk - Medyan: 1.07 dk — Std: 1.58 dk - Min: 0.20 dk — Maks: 64.88 dk - En yoğun saatler: 07:00-08:00 (2.918 sefer) ve 18:00-19:00 (2.734 sefer)

★ **BÜYÜK FIRSATA DÖNÜŞEN YENİLİK: GECİKME HESABI**

stop_times, GPS verisinden önce elimizde olan planlanan zaman çizelgesini verir.

GPS veri toplama başladıktan sonra her durak geçişi için şunu hesaplayabiliriz:

$$\text{GECIKME (saniye)} = \text{gerçek_varış_zamanı} - \text{planlanan_varış_zamanı} \quad (\text{stop_times.arrival_time})$$

Bu, makalede OLMAYAN ek bir hedef değişken seçeneği sunar:

Seçenek A (Makale ile aynı): Gerçek duraklar arası seyahat süresi (dakika)

Seçenek B (Özgün katkı): Gecikme (saniye/dakika) — planlanan seferden sapma

Gecikme tahmini daha zengin bir problem tanımı sağlar ve yolcu bilgilendirme sistemlerinde doğrudan kullanılabilir.

2.3 shapes.txt — Güzergah Geometrisi

350.858 GPS noktası, 788 şekil. Her hat için 2 şekil (gidiş + dönüş yönü) mevcuttur. trips.txt üzerinden her sefere hangi şeklin ait olduğu bilinmektedir.

- Bir sonraki durak arasındaki yol mesafesi hassas olarak hesaplanabilir
- Trafik arterleriyle güzergah kesişimi tespit edilebilir (hız-süre verisiyle birleştirme)
- Maks. şekil noktası: 2.107 (çok detaylı güzergah geometrisi)

2.4 calendar.txt — Gün Tipi Kesinleşti

Üç servis tipi birebir makale metodolojisiyle örtüşmektedir:

service_id	Günler	Sefer Sayısı (trips.txt)	Stop Times?
2917	Pazartesi – Cuma (Hafta İçi)	24.856 sefer	Evet — 22.934 sefer
2926	Cumartesi	22.671 sefer	Hayır — Toplanacak
2929	Pazar	19.870 sefer	Hayır — Toplanacak

Önemli Not: stop_times Sadece Hafta İçi (service_id=2917) İçeriyor

Mevcut stop_times.txt yalnızca hafta içi (Pazartesi-Cuma) seferlerini kapsıyor.

Cumartesi (2926) ve Pazar (2929) seferleri trips.txt'te tanımlı ama stop_times'ta planlanan zamanları yok.

Bu eksiklik GPS veri toplamayla tamamlanabilir veya hafta içi modeli önce kurulup ardından hafta sonu verisine genişletilebilir.

Başlangıç stratejisi: Hafta içi modeli kur, Cumartesi/Pazar için planlanan süreleri hafta içi ortalamasından türet.

3. GELİŞTİRİLMİŞ SİSTEM MİMARİSİ

3.1 Veri Akış Diyagramı

Tam Veri Akış Şeması
[API: hatotobuskonumları] ↓ Her 60 saniyede bir
[Ham GPS Log: bus_id, hat_no, koorX, koorY, timestamp] ↓ Sefer eşleşmesi (trips.txt üzerinden trip_id)
[Durak Ziyaret Kaydı] ↓ stop_times ile birleştirme
[Planlanan Süre + Gerçek Süre → Gecikme = Gerçek - Planlanan] ↓ Bağlamsal özellik ekleme
[Eğitime Hazır Dataset: 11 özellik + 1-2 hedef değişken] ↓ LSTM / GRU eğitimi
[Tahmin Modeli: gecikme veya varış süresi] ↓
[Demo: durag/{id} API → Tahminli varış süresi gösterimi]

3.2 Durak Eşleşme: Yeni Yaklaşım

V1.0 yaklaşımı (Haversine arama) artık gerekli değil. Yeni ve çok daha güvenilir süreç:

1. GPS kaydı alınır: (bus_id, hat_no, koorX, koorY, timestamp)
2. bus_id + hat_no ile trips.txt'ten aktif trip_id belirlenir
3. Bu trip_id için stop_times'tan tüm (stop_id, stop_sequence, arrival_time) listesi çekilir
4. Her durağın koordinatı stops.txt'ten alınır
5. GPS noktasının en yakın durak noktasına mesafesi hesaplanır (artık çok daha az hesap)
6. Otobüs bir durağa 50m'den fazla yaklaştığında ve uzaklaştığında 'durak geçildi' kaydedilir
7. Kaydedilen zaman - planlanan arrival_time = gecikme

Verimlilik Avantajı
V1.0: Her GPS noktası için 10.483 durakla mesafe hesabı → O(n) hesap, gürültülü sonuç
V2.0: Her GPS noktası için sadece ilgili seferin durakları kontrol edilir → tipik sefer 30-50 durak
Sonuç: ~200-300x daha hızlı eşleşme + çok daha güvenilir durak geçiş tespiti

4. GELİŞTİRİLMİŞ ÖZELLİK SETİ

Özellik	Kaynak Dosya	Tip	Açıklama	V1→V2
stop_sequence	stop_times.txt	Sayısal	Güzergah üzerindeki durak sıra no (maks 192)	Güçlendi
planned_travel_min ★	stop_times.txt	Sayısal	Planlanan duraklar arası süre — YENİ!	YENİ
elapsed_time_min	GPS log	Sayısal	Bir önceki duraktan geçen gerçek süre	Aynı
delay_so_far_min ★	GPS log – stop_times	Sayısal	Mevcut birikmiş gecikme — YENİ!	YENİ
remaining_stops	stop_times.txt	Sayısal	Hedefe kalan durak sayısı — YENİ!	YENİ
shape_distance_m ★	shapes.txt	Sayısal	Duraklar arası yol mesafesi (metre) — YENİ!	YENİ
day_type	calendar.txt (service_id)	Kategorik	0=Hafta içi, 1=Cumartesi, 2=Pazar	Kesinleşti
time_block	GPS timestamp	Kategorik	Sabah pik / Akşam pik / Normal / Gece	Aynı
weather_condition	Çekici hizmet verisi	Kategorik	Açık / Bulutlu / Kapalı / Yağmurlu	Aynı
road_condition	Çekici hizmet verisi	Kategorik	Kuru / Islak	Aynı
avg_speed_arterial	izbbhizsureverileri.xlsx	Sayısal	İlgili arterde saatlik ortalama hız	Aynı
traffic_volume	izbbizumanaarteraracsayimguncel.xlsx	Sayısal	İlgili arterde araç yoğunluğu	Aynı
direction_id	trips.txt	Kategorik	0=Gidiş, 1=Dönüş	Kesinleşti
trend_slope	Lineer regresyon	Sayısal	N<1000 grupta aktif (makale yöntemi)	Aynı

Hedef Değişken Seçenekleri

SEÇENEK A — Makale ile Aynı (Önerilen Başlangıç):

target = gerçek duraklar arası seyahat süresi (dakika)

SEÇENEK B — Özgün Katkı (Yayın Değeri Yüksek):

target = gecikme (saniye) = gerçek_varış – planlanan_varış

Pozitif = geç, Negatif = erken

SEÇENEK C — Kombine (En Kapsamlı):

İki modeli de kur, her ikisini de raporla

Gecikme tahmini uygulamada daha kullanışlı, makale karşılaştırması Seçenek A ile yapılır

5. GÜNCEL PROJE YOL HARİTASI

FAZA 1	Veri Toplama Altyapısı	2-3 Hafta
-----------	------------------------	-----------

1.1 GPS Veri Toplama Servisi

- Python APScheduler ile her 60 saniyede çalışan poller
- hatotobuskonumları/{hatId} — 15-20 öncelikli hat için konum akışı
- SQLite şeması: gps_log(bus_id, route_id, trip_id, stop_id, enlem, boylam, timestamp, planlanan_variş, gerçek_variş, gecikme_sn)
- Hata yönetimi: timeout, bağlantı kopması, duplicate önleme
- Toplama hızı: ~14.400 kayıt/gün (10 hat × 60sn × 24sa)

1.2 GTFS Entegrasyonu (Artık Hazır)

-
-
-
-
- Sefer tanımlama algoritması: aktif otobüsün hat_no + yön + saatine bakarak stop_times'tan trip_id belirleme

1.3 Keşifsel Veri Analizi

- Hafta içi sefer dağılımı: saatlik bazda yoğunluk haritası (pik: 07-08 → 2.918 sefer)
- stop_times planlanan süre dağılımı: ortalama 1.42 dk, max 64.88 dk (anormal sefer analizi)
- shapes.txt üzerinden en uzun / en kısa güzergahların haritalanması
- Hız-süre verisi ile trafik arterleri arasındaki GTFS güzergah kesişim haritası
- Biniş verisinden hat bazlı yolcu talep profili çıkarımı

FAZA 2	Veri Birleştirme ve Özellik Mühendisliği	2-3 Hafta
-----------	--	-----------

2.1 Durak Geçiş Kaydı — Yeni Yöntem

- GPS pozisyonu alındığında trip_id belirlenir
- stop_times'tan bu seferin sıradaki durağı ve koordinatı çekilir
- Otobüs durağa 50m'den yaklaştığında durak geçiş zamanı damgalanır
- gerçek_variş - planlanan_variş = gecikme hesaplanır ve kaydedilir
- Ardışık iki durak arası gerçek süre = birincil hedef değişken

2.2 shapes.txt ile Yol Mesafesi Hesabı

- Her durak çifti (stop_A → stop_B) için shapes.txt üzerinden Haversine kümülasyonu
- shape_distance_m özelliği oluşturulur (yeni, makalede yoktu)
- Km başına süre (dk/km) türev özelliği — gecikme anomali tespitinde kullanışlı

2.3 Bağlamsal Özellik Birleştirme

- Timestamp → calendar.txt üzerinden service_id → day_type (kesin)
- Timestamp → zaman dilimi (sabah pik 07-09 / akşam pik 17-19 / normal / gece)
- GPS zamanı + ilçe → çekici hizmet verisiyle hava/yol durumu eşleştirme
- GPS zamanı + güzergah artere yakınlık → hız-süre verisiyle trafik eşleştirme
- Trend özelliği: (hat × hava × zaman_dilimi) grubu N<1000 ise aktif

FAZA
3

Model Geliştirme ve Eğitimi

3–4 Hafta

3.1 Hedef Değişken A: Gerçek Seyahat Süresi

- Makaleyle doğrudan karşılaştırılabilir
- Girdi: [stop_sequence, planned_travel_min, elapsed_time_min, day_type, time_block, weather, road, speed, traffic, shape_distance_m, trend]
- Hedef: gerçek duraklar arası seyahat süresi (dakika)

3.2 Hedef Değişken B: Gecikme Tahmini (Özgün Katkı)

- Ek özellikler: delay_so_far_min, remaining_stops, planned_travel_min
- Hedef: gecikme (saniye) — negatif = erken, pozitif = geç
- Pratik uygulama: yolcuya 'Bu otobüs X saniye geç' mesajı

3.3 Model Karşılaştırma Matrisi

Model	Varyant	Beklenti	Öncelik
LSTM	Trend Yok	Ana baseline	Zorunlu
LSTM	Trend Var (N<1000)	En iyi performans	Zorunlu
GRU	Trend Yok	Hızlı alternatif	Zorunlu
GRU	Trend Var	Karşılaştırma	Zorunlu
Random Forest	—	Makale Tablo 11 kıyası	Önerilen
XGBoost	—	Makale Tablo 11 kıyası	Önerilen
Transformer	—	Veri yeterliyse test	Opsiyonel

3.4 Eğitim Protokolü

- Bölünme: trip_id × servis_tarihi bazında %70 / %15 / %15
- Normalizasyon: MinMaxScaler, sadece eğitim setinden
- Kayıp: MAE (birincil), MSE (ikincil)
- Optimizer: Adam, lr=0.001
- Erken durdurma: sabır=5, doğrulama MAE izle
- Epoch: 50, batch: 64

FAZA
4

Değerlendirme ve Analiz

2 Hafta

4.1 Koşul Bazlı Analiz (Makale ile Aynı Çerçeve)

- Zaman dilimine göre: Sabah pik / Akşam pik / Normal / Gece
- Hava koşuluna göre: Açık / Bulutlu / Islak Yol
- Gün tipine göre: Hafta içi (2917) / Cumartesi (2926) / Pazar (2929)
- Örneklem boyutuna göre: 0-1K / 1K-2K / 2K-3K / 3K+

4.2 Yeni: Gecikme Analizi

- Hat bazlı kronik gecikme profili: Hangi hatlar sürekli geç?
- Durak bazlı gecikme birikimi: Gecikme güzergah boyunca nasıl artıyor?
- Trafik arterleri ile gecikme korelasyonu (hız-süre verisi ile birlikte)
- Yağmurlu vs Kuru yol: gecikme etkisinin sayısal tespiti

4.3 İstatistiksel Doğrulama

- 95% güven aralıkları (Monte Carlo Dropout — makaleyle aynı)
- Diebold-Mariano testi: LSTM vs GRU istatistiksel anlamlılık
- SHAP analizi: planned_travel_min, time_block, shape_distance_m, weather önem sıralaması

FAZA

5

Demo Uygulama ve Raporlama

2 Hafta

5.1 Demo Uygulaması (Streamlit / Flask)

- Durak seç → Yaklaşan otobüsler (duragayaklasanotobusler API) + tahminli gecikme
- Hat seç → Tüm otobüslerin konumu haritada + planlanan vs gerçek karşılaştırma
- İnteraktif görsel: Saatlik gecikme dağılımı, hava etkisi, hat performansı

5.2 Raporlama Çıktıları

- Makaleyle birebir kıyaslanabilir Tablo 5, 6, 7, 8, 9, 10 benzerleri
- Özgün katkı: Gecikme analizi ve planlı süre özelliğinin model performansına etkisi
- shapes.txt tabanlı yol mesafesi özelliğinin SHAP önem analizi
- İzmir – İstanbul karşılaştırmalı bulgular

6. VERİ TOPLAMA BAŞLANGIÇ STRATEJİSİ

HEMEN BAŞLANMASI GEREKEN ADIMLAR

1. stop_times.txt'i bir SQLite veritabanına yükle — bu planlanan zaman tabanının
2. trips.txt + routes.txt + stops.txt'i de aynı DB'ye al
3. calendar.txt'ten tarih → service_id fonksiyonu yaz
4. API poller scriptini çalıştır (aşağıdaki hat listesiyle)
5. Her GPS kaydında: trip_id bul → stop_times'tan planlanan zamanı çek → gecikmeyi hesapla → kaydet

6.1 Öncelikli Hat Listesi

stop_times.txt verilerine göre en çok kayıt içeren (dolayısıyla en zengin planlama verisi olan) hatlar:

Hat No	stop_times Kayıt Sayısı	Ort. Planlanan Süre (dk)	Öncelik
975	18.130	0.87	★★★
775	8.854	0.99	★★★
984	7.889	1.25	★★★
776	7.379	1.51	★★★
701	7.148	0.90	★★★
712	6.969	1.08	★★
879	6.930	1.35	★★
171	6.789	1.46	★★
285	6.097	1.23	★★
835	5.544	2.00	★★

6.2 Günlük Tahmini Veri Birikimi

Süre	GPS Kaydı	Sefer-Durak Geçişi	Model İçin Yeterlilik
1 Hafta	~100.800 kayıt	~35.000 geçiş	Ön analiz için yeterli
2 Hafta	~201.600 kayıt	~70.000 geçiş	Baseline model denenebilir
1 Ay	~432.000 kayıt	~150.000 geçiş	Makul model eğitimi
2 Ay	~864.000 kayıt	~300.000 geçiş	İyi model — makale miktarına yaklaşıyor

7. MAKALEYİ AŞAN ÖZGÜN KATKILAR

Konu	Makale (Kaya & Kalay, 2025)	Projemiz	Fark
Planlanan Süre	Özellik olarak KULLANILMADI	planned_travel_min özellik olarak eklendi	Bizde avantaj
Gecikme Analizi	Sadece tahmin hatası (MAE/RMSE)	Gecikme hedef değişken olarak modellendi	Özgün katkı
Güzergah Geometrisi	GTFS stop_sequence (dolaylı)	shapes.txt + yol mesafesi (metre)	Daha zengin
Gün Tipi	Timestamp'ten türetme	calendar.txt service_id — kesin kaynak	Daha güvenilir
Trafik Verisi	Sadece hava + GTFS	Hız-süre + araç sayım verisi eklendi	Bizde avantaj
Biniş Talebi	Özellik olarak KULLANILMADI	Günlük biniş verisi (2015-2024) mevcut	Eklenebilir fark
Şehir	İstanbul (İETT)	İzmir (ESHOT)	Bağımsız doğrulama

8. GÜNCELLENMİŞ RİSKLER VE ÇÖZÜMLER

Risk	Açıklama	Çözüm
stop_times eksikliği (Cts/Paz)	service_id 2926-2929 için planlanan zaman yok	Hafta içi ortalamasından türet veya önce hafta içi modeli kur
Trip_id belirsizliği	Gerçek zamanlı otobüsün hangi trip'e ait olduğu her zaman net değil	Hat + yön + saat ile stop_times'tan en yakın trip_id eşleştirme
GPS veri boşlukları	API kesintilerinde seferler takip edilemez	Kesinti log'u tut, eksik geçişleri işaretle, model eğitiminde filtrele
Planlanan süre anomalileri	stop_times'ta 64.88 dk'lık aşırı değerler mevcut	10 dk üzeri segmentleri ayrı kategoride değerlendir veya filtrele
Hava verisi eksikliği	Çekici hizmet verisi tüm tarihleri kapsamıyor	OpenWeatherMap API ile tamamla